**Academic Year: 2026-27**
**Semester: VII**
**Class / Branch/ Div: BE- IT/B**
**Subject: SAD Lab**
**Name of Instructor:Prof. Ganesh Gourshete**

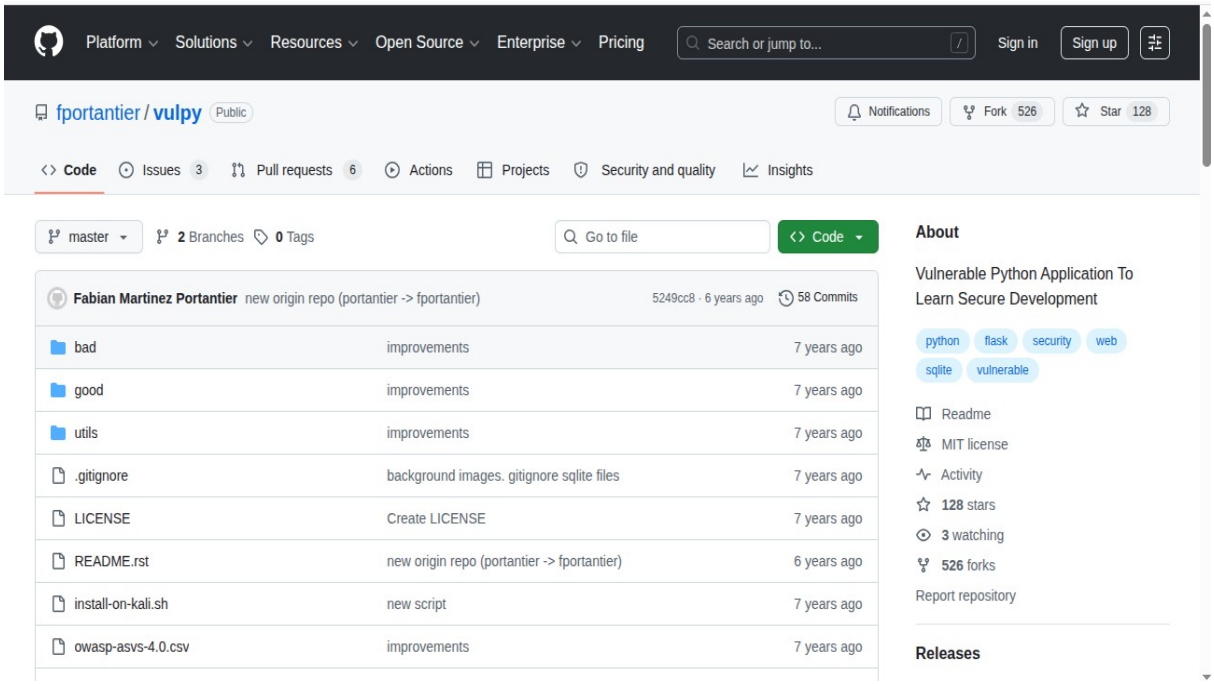
**Name of Student: Sonika Sawant**
**Student ID:23104054**
**Roll No.65**
**Date of Submission:21-07-26**

# Experiment No:3

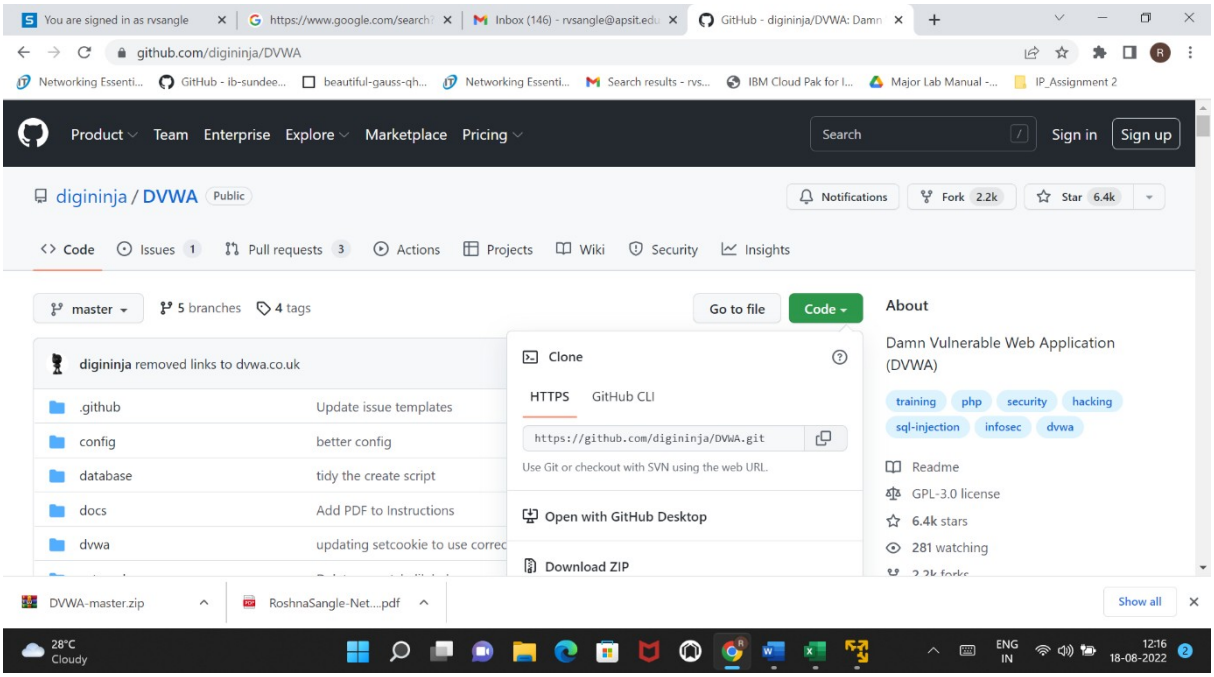
**Aim:** To study and demonstrate the use of SAST Tool.

**Step 1:** To get sample vulnerable code of python code use GitHub as shown below.

**Example :- vulpy**



**Step 2:** Either download the sample code zip file from GitHub of copy folder link as shown below.




**Step 3:** Once sample python code is copied make its clone using following command in terminal.

```
apsit@apsit-HP-280-G3-MT:~$ git clone https://github.com/fportantier/vulpy.git
Cloning into 'vulpy'...
remote: Enumerating objects: 437, done.
remote: Total 437 (delta 0), reused 0 (delta 0), pack-reused 437 (from 1)
Receiving objects: 100% (437/437), 2.90 MiB | 4.46 MiB/s, done.
Resolving deltas: 100% (223/223), done.
apsit@apsit-HP-280-G3-MT:~$
```

**Step 4:** Now check the files present in path using ls.

```
roshna@ubuntu:~/vulpy$ ls
bad  good  install-on-kali.sh  LICENSE  owasp-asvs-4.0.csv  README.rst  requirements.txt  utils  Vulnerable-Code-Snippets
```

**Step 5:** Now to get path of vulpy use pwd command.

```
apsit@apsit-HP-280-G3-MT:~/vulpy/bad$ pwd
/home/apsit/vulpy/bad
apsit@apsit-HP-280-G3-MT:~/vulpy/bad$
```

**Step 6:** Now test the given python code with bandit as shown below.

```
--------------------------------------------------

Code scanned:
        Total lines of code: 495
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0.0
                Low: 3.0
                Medium: 0.0
                High: 2.0
        Total issues (by confidence):
                Undefined: 0.0
                Low: 0.0
                Medium: 2.0
                High: 3.0
Files skipped (0):
apsit@apsit-HP-280-G3-MT:~/vulpy/bad$
```

| Bandit Error | Issue ID | Severity | Location | Cause | Recommended Fix |
|---|---|---|---|---|---|
| Importing `subprocess` | **B404** | Low | `brute.py:3` | The `subprocess` module can execute external programs, which may introduce security risks if used improperly. | Avoid `subprocess` if a Python library can perform the task. If it is required, validate all inputs and restrict executable paths. |
| `subprocess.run()` with external input | **B603** | Low | `brute.py:21` | Executing a program using variables ( `program` , `username` , `password` ) may lead to execution of untrusted input. | Validate and sanitize all user-controlled inputs. Use absolute executable paths, avoid passing untrusted values, and consider allow-listing permitted commands. |
| Empty `except` block ( `except: pass` ) | **B110** | Low | `libsession.py:21` | Exceptions are silently ignored, making debugging difficult and potentially hiding security issues. | Catch specific exceptions (e.g., `ValueError` , `json.JSONDecodeError` ) and log or handle them appropriately instead of using `pass` . |
| Flask running with `debug=True` | **B201** | High | `vulpy-ssl.py:29` | Flask debug mode exposes the interactive debugger, which can allow arbitrary code execution if accessible. | Disable debug mode in production: `app.run(debug=False, host='127.0.0.1', ssl_context=(...))` or configure debug via environment variables ( `FLASK_DEBUG=0` ). |
| Flask running with `debug=True` | **B201** | High | `vulpy.py:55` | Same issue as above. The Werkzeug debugger is enabled. | Change to `app.run(debug=False, host='127.0.0.1', port=5000)` or control debug mode through application configuration. |

**Conclusion:**

Code should be clean and safe. Vulnerabilities and bugs in software under development constitute a major security problem. Thus, we have mitigated similar kind of security risk associated with Python code using bandit as a SAST tool efficiently.